



ENSEA

Beyond Engineering

SYSTÈMES À MICROCONTRÔLEUR

Compte Rendu TP5 –
Liaisons et bus, consommation
électrique

1. Mesure de la consommation électrique

1.1. Préparation

Pour mesurer un courant, on peut utiliser deux méthodes :

- Mesure « Low-Side » :
On place une résistance de shunt entre la charge de courant à mesurer et la masse. Cette méthode a pour avantage d'être facile à mettre en place (la carte peut facilement mesurer la différence de tension entre la masse ayant un potentiel nul et la résistance).
En contrepartie, comme la charge traverse une résistance avant d'atteindre la masse, la tension de référence pour le reste du circuit n'est plus tout à fait à 0 V ce qui peut créer des problèmes notamment en essayant de communiquer avec d'autres systèmes qui ont une référence qui est réellement à 0 V.
- Mesure « High-Side » :
A l'inverse, on peut placer le shunt entre la source de tension et la charge. Puisque la charge atteint directement la masse, la référence du circuit est réellement à 0 V, il n'y a plus les perturbations vues précédemment.
Cette méthode présente cependant l'inconvénient de nécessiter un amplificateur spécifique capable de mesurer une faible différence de tension entre deux potentiels proches, puisque le shunt ne va créer qu'une faible différence de potentiel.

On peut également mentionner la méthode par effet Hall qui permet d'estimer une charge grâce au champ magnétique qu'elle génère autour d'elle. Cela présente l'avantage de ne pas perturber le fonctionnement du circuit puisque la mesure se fait à l'extérieur de celui-ci, mais cette méthode est beaucoup plus coûteuse et encombrante, et est surtout moins précise pour les faibles courants comme ceux utilisés sur les cartes électroniques à notre disposition.

Sur la figure 9.2 de l'énoncé, on constate que l'amplificateur mesure le courant entre *Load* et *Supply* qui alimentent « la carte Nucleo et les éléments passifs connectés (notamment les LEDs) » d'après l'énoncé. Ainsi, on ne mesure pas en revanche la consommation du moteur pas à pas ni de l'écran OLED.

Une carte Nucleo ne consomme que quelques milliampères. Ainsi selon l'utilisation ou non des LEDs, on peut estimer l'ordre de grandeur du courant I_{load} entre 10 et 50 mA (on prendra $I_{load} \cong 30 \text{ mA}$)

Sachant que la résistance vaut $R_{shunt} = 30 \text{ m}\Omega$, on a alors :

$$V_{shunt} = R_{shunt} \times I_{load}$$

AN: $V_{shunt} \cong 30 \cdot 10^{-3} \times 30 \cdot 10^{-3} = 900 \text{ }\mu\text{V}$

On peut en déduire la tension en sortie de l'amplificateur V_{out} grâce à la formule donnée dans la figure 9.1 :

$$V_{out} = (I_{load} \times R_{shunt}) \times Gain + V_{Ref}$$

Sachant qu'il est indiqué sur cette même figure que le gain est de 200, et que la tension de référence est reliée à la masse (donc $V_{Ref} = 0 \text{ V}$), on a alors :

AN: $V_{out} \cong (30 \cdot 10^{-3} \times 30 \cdot 10^{-3}) \times 200 + 0 = 0,18 \text{ V}$

L'ADC étant codé sur 12 *bits*, il peut prendre des valeurs allant de 0 à 4095.

Pour obtenir l'ordre de grandeur de la valeur de l'ADC, sachant que la tension du circuit vaut $V_{cc} = 3,3 \text{ V}$, on applique la formule :

$$ADC = \frac{V_{out}}{V_{cc}} \times 4095$$

AN: $ADC \cong \frac{0,18}{3,3} \times 4095 = 223$

1.2. Manipulation de base

Dans la fonction **Loop** écrite dans le fichier `mesure_consommation_courant.c` fourni, la ligne de code venant mettre la carte en « Sleep Mode » n'est pas commentée contrairement à ce qui est affiché dans l'énoncé. Étant donné qu'il nous est demandé dans la partie suivante d'ajouter un mode d'économie d'énergie, on partira donc dans ce compte rendu du principe que l'énoncé était correct et on corrige la fonction **Loop** pour qu'elle soit vide.



COM7 - PuTTY			
Mesure courant :	2337	1.883297	23.541210
Mesure courant :	2279	1.836557	22.956959
Mesure courant :	2337	1.883297	23.541210
Mesure courant :	2299	1.852674	23.158424
Mesure courant :	2306	1.858315	23.228937
Mesure courant :	2325	1.873626	23.420330
Mesure courant :	2299	1.852674	23.158424
Mesure courant :	2291	1.846227	23.077839
Mesure courant :	2332	1.879267	23.490843
Mesure courant :	2319	1.868791	23.359890

Fig. 1 et 2 – Mesure de I_{mes} à l'oscilloscope et affichage de la console (LEDs éteintes)



COM7 - PuTTY			
Mesure courant :	3768	3.036484	37.956047
Mesure courant :	3757	3.027619	37.845238
Mesure courant :	3757	3.027619	37.845238
Mesure courant :	3773	3.040513	38.006409
Mesure courant :	3736	3.010696	37.633701
Mesure courant :	3747	3.019560	37.744503
Mesure courant :	3782	3.047765	38.097069
Mesure courant :	3770	3.038095	37.976189
Mesure courant :	3725	3.001832	37.522896
Mesure courant :	3725	3.001832	37.522896

Fig. 3 et 4 – Mesure de I_{mes} à l'oscilloscope et affichage de la console (LEDs allumées)

On note une incohérence entre le schéma donné dans la préparation et le code fourni.

En effet la figure 9.1 de l'énoncé indique que la résistance utilisée est de $30\text{ m}\Omega$ tandis que la valeur utilisée à la place de la résistance lors du calcul de *intensity* dans la fonction **HAL_ADC_ConvCpltCallback** est **0.4**, raison pour laquelle la valeur affichée dans la console pour la tension lorsque les LEDs sont éteintes est d'environ 1.8 ce qui est différent du $V_{out} \cong 0,18\text{ V}$ calculé dans la préparation.

Puisque la mesure réelle à l'oscilloscope est également à $1,8\text{ V}$ en moyenne, on en conclut que la valeur réelle de la résistance est bien de $0,4\text{ }\Omega$ et non pas $30\text{ m}\Omega$ et donc que la valeur affichée dans la console est correcte.

On constate bien une montée de la tension mesurée à l'oscilloscope et calculée par la carte lorsque les LEDs sont allumées.

On observe également que la tension n'est pas constante mais ressemble plutôt à un signal carré : cela est dû aux interruptions régulières par la fonction `HAL_ADC_ConvCpltCallback` qui appelle la fonction `printf` et augmente donc la consommation du processeur.

1.3 Mode power down

Le « Sleep Mode » coupe l'horloge du processeur, mais les horloges et les périphériques sont toujours en marche, ce qui permet de réveiller le processeur lors d'une interruption. Cela permet d'économiser de l'énergie en attendant la fin du travail d'un des périphériques par exemple.

Le « Stop Mode » vient couper les oscillateurs, ce qui arrête le processeur et la plupart des périphériques. La mémoire stockée dans la RAM est conservée ce qui permet de reprendre le programme là où il a été arrêté, puisqu'on peut redémarrer les horloges à l'aide d'une interruption externe. Il faut tout de même compter un certain temps d'attente pour réaliser ce redémarrage.

Enfin, le « Standby Mode » coupe complètement l'alimentation. On perd la mémoire RAM et le seul moyen de redémarrer et d'utiliser le bouton « Reset » qui vient redémarrer complètement la carte et le programme.

Dans notre cas, le mode le plus pertinent est le « Sleep Mode » puisqu'il permet aux horloges de continuer de tourner et donc de réaliser l'interruption périodique permettant d'afficher les valeurs de *ADC* et de la tension et l'intensité qui en sont déduit, le tout de façon autonome.

On ajoute ce mode dans la fonction `loop` et on constate une nette diminution des valeurs mesurées (voir figures pages suivantes).



COM7 - PuTTY			
Mesure courant :	1455	1.172527	14.656593
Mesure courant :	1461	1.177363	14.717033
Mesure courant :	1463	1.178974	14.737180
Mesure courant :	1464	1.179780	14.747253
Mesure courant :	1438	1.158828	14.485347
Mesure courant :	1435	1.156410	14.455127
Mesure courant :	1435	1.156410	14.455127
Mesure courant :	1435	1.156410	14.455127
Mesure courant :	1466	1.181392	14.767400
Mesure courant :	1448	1.166886	14.586081

Fig. 5 et 6 – Mesure de I_{mes} à l'oscilloscope et affichage de la console avec arrêt complet du processeur lorsqu'il n'est pas sollicité (LEDs éteintes)



COM7 - PuTTY			
Mesure courant :	2863	2.307179	28.839743
Mesure courant :	2876	2.317656	28.970694
Mesure courant :	2935	2.365201	29.565018
Mesure courant :	2891	2.329744	29.121796
Mesure courant :	2874	2.316044	28.950550
Mesure courant :	2901	2.337802	29.222528
Mesure courant :	2863	2.307179	28.839743
Mesure courant :	2863	2.307179	28.839743
Mesure courant :	2877	2.318462	28.980770
Mesure courant :	2887	2.326520	29.081503

Fig. 7 et 8 – Mesure de I_{mes} à l'oscilloscope et affichage de la console avec arrêt complet du processeur lorsqu'il n'est pas sollicité (LEDs allumées)

2. Liaisons et bus

2.1 Le composant BMM150

On complète la fonction **readOneRegister** :

```
uint8_t readOneRegister(uint8_t adress){  
    // A modifier : Le MSB de l'adresse doit forcément valoir 1.  
    uint8_t dataToSend = adress | 0x80;  
    uint8_t receiveBuffer=0;  
  
    // Mise à 0 de la ligne de chip select  
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_RESET);  
  
    // Transmission de l'adresse modifié dans dataToSend  
    HAL_SPI_Transmit(localSPIHandler, &dataToSend, 1, 10);  
  
    // Réception d'un seul octet  
    HAL_SPI_Receive(localSPIHandler, &receiveBuffer, 1, 10);  
  
    // Mise à 1 de la ligne de chip select.  
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_SET);  
  
    return receiveBuffer;  
}
```

Puis on modifie la fonction **setup** pour initialiser le driver :

```
extern SPI_HandleTypeDef hspi2;  
  
void setup(){  
    printf("**** TP5 : liaisons et bus, consommation electrique**** \r\n");  
    HAL_TIM_Base_Start(&htim4);  
    HAL_ADC_Start_IT(&hadc1);  
    initDriverBMM150(&hspi2);  
}
```

Le code ne renvoie pas d'erreur lors de la compilation et semble donc fonctionner.

Lors de la visualisation à l'analyseur logique, on obtient l'affichage suivant sur l'oscilloscope :

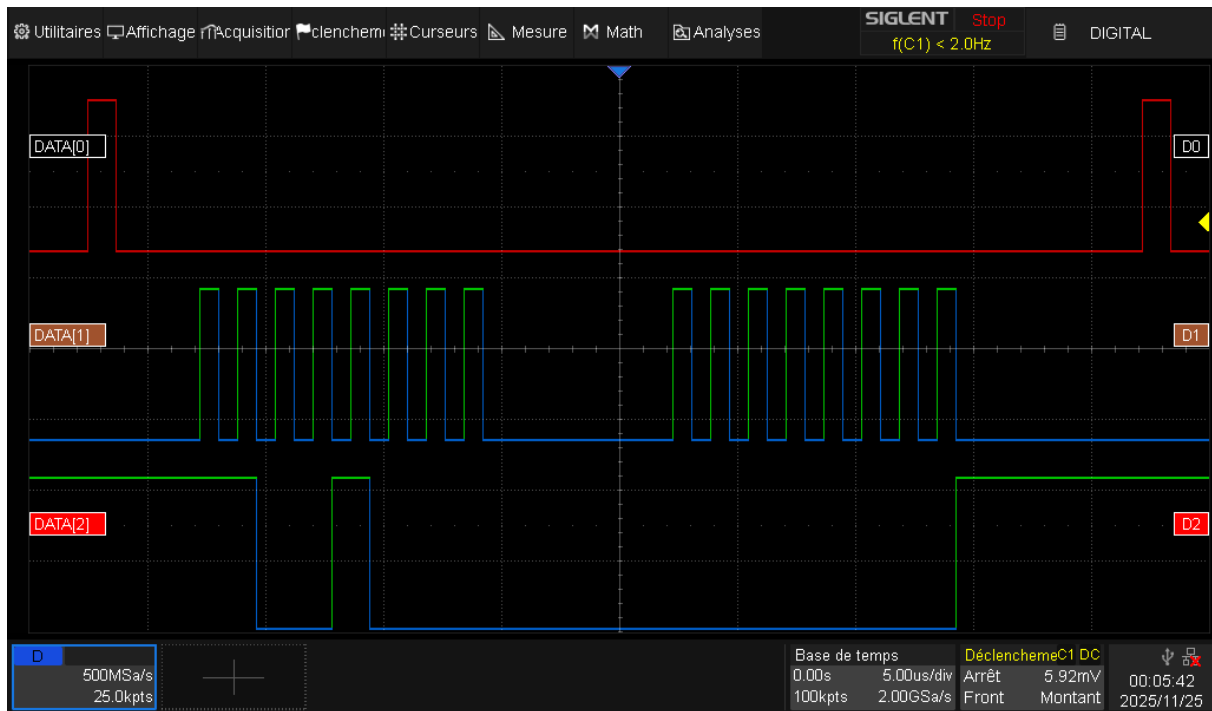


Fig. 9 – Affichage du transfert sur l'oscilloscope grâce à un analyseur logique

La ligne CS (ici en rouge) encadre parfaitement le transfert en passant à l'état bas. Sur la ligne MOSI (ici en verte), on observe l'octet `0xC0` (11000000) lors de la phase d'adressage. Cela confirme que la fonction `readOneRegister` ajoute correctement le bit de lecture (MSB à 1) à l'adresse de base du Chip ID (`0x40`)

2.2 Détection de l'angle par rapport au Nord magnétique

On complète la fonction **readManyRegister** :

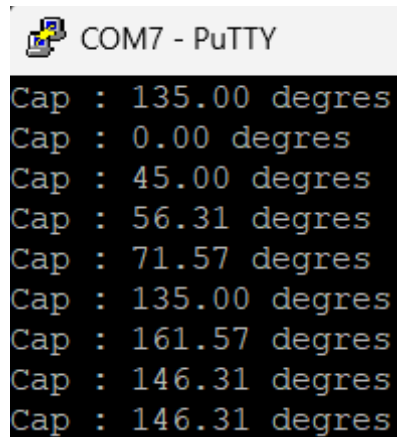
```
void readManyRegister(uint8_t address, int8_t * pData, uint8_t size){  
    // A modifier : Le MSB de L'adresse doit forcément valoir 1.  
    uint8_t dataToSend = address | 0x80;  
  
    // Mise à 0 de la ligne de chip select  
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_RESET);  
  
    // Transmission de L'adresse modifié dans dataToSend (1 seul octet)  
    HAL_SPI_Transmit(localSPIHandler, &dataToSend, 1, 10);  
  
    // Réception de size octets  
    HAL_SPI_Receive(localSPIHandler, (uint8_t*)pData, size, 10);  
  
    // Mise à 1 de la ligne de chip select.  
    HAL_GPIO_WritePin(SPI_CS_GPIO_Port, SPI_CS_Pin, GPIO_PIN_SET);  
}
```

Puis on modifie la fonction **loop** en enlevant le passage en « Sleep Mode » et en ajoutant une mesure et un affichage de l'angle dans la console :

```
void loop(){  
    //HAL_PWR_EnterSLEEPMode(0,PWR_SLEEPENTRY_WFI);  
  
    //Lecture de L'angle (fonction bloquante car elle attend le bit DRDY)  
    float angle_rad = readAngle();  
  
    //Conversion Radians -> Degrés (180 / PI = 57.296)  
    float angle_deg = angle_rad * 57.296f;  
  
    //Affichage  
    printf("Cap : %.2f degrees\r\n", angle_deg);  
}
```

(on pense également à passer le contenu de **HAL_ADC_ConvCpltCallback** en commentaire afin d'enlever l'affichage de la consommation électrique, comme suggéré dans l'énoncé)

Finalement on obtient un affichage de l'angle dans la console qui est cohérent lorsque la carte est en mouvement.



```
COM7 - PuTTY
Cap : 135.00 degrees
Cap : 0.00 degrees
Cap : 45.00 degrees
Cap : 56.31 degrees
Cap : 71.57 degrees
Cap : 135.00 degrees
Cap : 161.57 degrees
Cap : 146.31 degrees
Cap : 146.31 degrees
```

Fig. 9 – Affichage du transfert sur l'oscilloscope grâce à un analyseur logique

Bien que le temps ne nous l'ai pas permis, nous avons également pensé à ajouter une variable `static` dans la fonction `loop` afin de garder en mémoire le dernier angle mesurer et ainsi actualiser la console seulement lorsque la carte détecte un mouvement, pour ne pas avoir un flux de texte illisible.